Kansei Language Reference

Kansei to be an intuitive scripting language designed for simplicity and flexibility. Inspired by Ruby 1.8 and functional programming.

Comments

```
# This is a comment
x = 10 # Inline comment
```

Data Types

Primitives

- **Integer**: `1`, `42`, `-10`
  - Default integer literals are `i64`.
  - Use suffixes like `1i32`, `1i64`, `1i128` for signed sizes.
  - Use suffixes like `1u32`, `1u64`, `1u128` for unsigned sizes.
- **Float**: `1.0`, `3.14`, `-0.01`
  - Default float literals are `f64`.
  - Use suffixes like `1.0f32`, `1.0f64`, `1.0f128` to pick other float sizes.
- **String**: `"Hello"`, `"World"`
- **Boolean**: `true`, `false`
- **Nil**: `nil`

Data Structures

Arrays and Maps are **mutable** and use **reference semantics**. Assigning an array/map to a new variable does not copy it; both variables point to the same data.

- **Array**: Ordered list of values.

```
a = [1, 2, 3]
first = arr[0]

# Modification
arr[0] = 10

# Initialization with size and value
zeros = [0; 10]

# Initialization with generator function
evens = [fn(i) i * 2 end; 5] # [0, 2, 4, 6, 8]
evens = [{|i| i * 2 }; 5] # same

a = [1, 2, 3]
copy = clone a
a each  { |i| i + 1 } # [2, 3, 4]
a apply { |i| i + 1 } # [1, 2, 3] original input
```

Specialized numeric arrays are created when all elements are numeric and share a compatible type:

  - **I32Array**: `i32`/`u32` values (e.g., `[1i32, 2u32]`, `[0i32; 10]`)
  - **I64Array**: integer values (default for integer-only lists)
  - **F32Array**: `f32` values (e.g., `[1.0f32, 2.0f32]`, `[0.0f32; 10]`)
  - **F64Array**: floating-point values (default for float-only lists)

Numeric array literals and generators will choose the most specific type that matches all elements. These specialized arrays support indexing, slicing, `len`, and iteration like `Array`, but store their elements in a compact numeric representation.

- **Map**: Key-value pairs (keys are strings).

```
user = {"name": "Alice", "age": 30}
name = user["name"]

# Modification
user["age"] = 31
user.age = 32 # Dot syntax works for assignment too

keys = user keys
values = user values

# Iterate
h = {"a": 1, "b": 2}
h keys each { |k, &h| h[k] = h[k] + 1 }  # [2, 3]
h keys apply { |k, &h| h[k] = h[k] + 1 } # ["a", "b"] original input
```

Use `each` or `apply` with blocks to iterate. `each` returns an array of block results, while `apply` is for side effects and returns the original collection.

Dot Syntax

Maps can be accessed and modified using dot notation if the key is a valid identifier.

```
user = {"name": "Alice"}
user.name = "Bob"
```

Program Arguments

Command-line arguments are available via the global `program` object.

```
# The name of the executable (args[0])
name = program.name

# The script arguments
args = program.args
puts args[0] # "arg1"

# Exit with optional code (defaults to 0)
program.exit()
program.exit(1)
```

Environment Variables

Environment variables are available via `program.env`.

```
home = program.env.HOME
```

Modules, `use`, and `import`

Kansei exposes native modules via the `std` namespace. The `use` keyword validates that a module path exists but does not create local bindings. Use assignment to alias. Use `@file` or `@function` with `use/import/load` when you want those bindings visible to nested functions.

`std::lib` modules are feature-gated in the Rust build. By default, all std::lib modules are enabled. To disable a module, build without defaults and opt in to the ones you want:

```
cargo build --no-default-features --features lib-math,lib-regex
```

std

The `::` operator accesses module members, similar to map dot access.

std::f64(x), std::f32(x), std::i64(x), std::i32(x)`, std::u64(x), std::u32(x)

- `std::f64(x)`, `std::f32(x)`, `std::i64(x)`, `std::i32(x)`, `std::u64(x)`, `std::u32(x)` -> casts These cast helpers are also availableo as globals (`f64(x)` etc). Integer casts do not accept floats (use rounding) Out-of-range values will error. These work also with strings, e.g. `f64("1.0")`

std::Int64 std::Int128 std::Uint64 std::Uint128 std::Float32 std::Float64 std::Float128

Somewhat redundant to the shortform modules (e.g. std::f64() but many more operations.

```
use std::Int64
use std::Int128
use std::Uint64
use std::Uint128
use std::Float32
use std::Float64
use std::Float128

Int64 = std::Int64
value = Int64.parse("42")
Int128 = std::Int128
big_int = Int128.parse("9007199254740993")
Uint64 = std::Uint64
u = Uint64.parse("42")
Float32 = std::Float32
f = Float32.parse("1.25")
root32 = Float32.sqrt(9)
Float64 = std::Float64
pi = Float64.parse("3.14159")
root64 = Float64.sqrt(9)
Float128 = std::Float128
big = Float128.parse("1.2345678901234567")
root128 = Float128.sqrt(9)

use std::simd simd = std::simd
simd.sum([1,2,3,4])  # -> 10
```

std::parallel

`std::parallel` provides parallel helpers backed by Rayon. It supports both native functions and user-defined Kansei functions/closures. User-defined functions are executed in isolated, thread-local interpreters.

Note: Functions passed to `std::parallel` must be **self-contained**. They cannot access variables from the outer scope via implicit or explicit capture (except via the provided `context` argument). They should only use their parameters and local logic.

```
use std::parallel
parallel = std::parallel

# Using a closure with parallel.collect
# Preferred: parallel.collect(count, context, function)
# The closure receives (context, index) as arguments.
n = 10

# Env context fields are automatically injected into the environment!
```

```
# You can access 'a' directly.
# The Env object itself is passed as the first argument 'ctx'.
context = %{ "a": 5 }
results = parallel.collect(n, context, {|ctx, i| i + a })
# -> [5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
# If context is a Map or Struct, fields are not injected; use ctx.

# Thread Safety and Data Copying
# Data passed to parallel threads (including `context`) is COPIED by value.
# Shared mutability is NOT supported.
# If you pass a reference `&x` as context, it is automatically dereferenced and its
VALUE is copied.
# Therefore, `&ref` semantics do not apply across threads; modifications are thread-
local.
x = 10
parallel.loop(5, {|i, ctx|
  # ctx is a local copy of x's value (10)
  # Modifying it here has no effect on 'x' in the main thread.
}, &x)
```

Available functions:

- `std::parallel::map(array, fn)` -> array of results
- `std::parallel::each(array, fn)` -> array of results
- `std::parallel::apply(array, fn)` -> original array (used for side effects)
- `std::parallel::loop(count, fn, context = nil)` -> nil (side effects only)
- `std::parallel::collect(count, fn, context = nil, into = nil)` -> array of results

std::kansei

Interpreter related modules.

std::kansei::ast

`std::kansei::ast` exposes AST and S-Expr helpers for tooling and metaprogramming.

```
use std::kansei
ast = std::kansei::ast

sexpr = ast.to_sexpr("a = 1 + 2")
ast = ast.from_sexpr(sexpr)
source = kansei.ast.to_source(ast)

value_sexpr = kansei.value.to_sexpr([1i32, 2i32])
value = kansei.value.from_sexpr(value_sexpr)
```

Available functions:

- `std::kansei::ast::to_sexpr(src_or_ast)` -> S-Expr string
- `std::kansei::ast::from_sexpr(sexpr)` -> `Ast`
- `std::kansei::ast::to_source(src_or_ast)` -> canonical source string
- `std::kansei::ast::from_source(src)` -> `Ast`
- `std::kansei::ast::eval_in(ast_or_src, env, program_or_nil)` -> value

`eval_in` evaluates the AST in a fresh environment populated from a frozen `Env`. You can pass an `Env` directly or a Map/Struct (it will be frozen). No standard library is injected unless you pass it in. Pass `&program` to expose the program object, or `nil` to omit it.

```
use std::kansei ast = std::kansei::ast

env = %{ "x": 3 }
ast.eval_in("x + 2", env, nil)    # -> 5
ast.eval_in("program.name", env, &program)
# To use std inside eval_in, include it explicitly: env = %{ "std": std, "x": 3 }
```

std::kansei::value

`std::kansei::value` exposes AST and S-Expr helpers for tooling and metaprogramming.

```
use std::kansei value = std::kansei::value
=> {"to_sexpr": <native function>", "from_sexpr": <native function>"}
k> value::to_sexpr([1,2,3])
=> "(i64array 1 2 3)"
k> a_s = value::to_sexpr([1,2,3])
=> "(i64array 1 2 3)"
k> a = value::from_sexpr(a_s)
=> [1, 2, 3]
```

- `std::kansei::value::to_sexpr(value)` -> S-Expr string
- `std::kansei::value::from_sexpr(sexpr)` -> value
- `std::kansei::value::inspect(value)` -> inspect string

Structs

```
struct Point
{
  x: Float64,
  y: Float64
}

fn sum(p { x: Float64, y: Float64 })  # matches structs having x and y
  p.x + p.y
end

p = Point { x: 1.0, y: 2.0 }
p.x = 3
puts sum(p)


fn Point.sum(self)      # only works on Point, needs self, cannot shadow a field
  self.x + self.y
end

puts p.sum()
```

File modules with `import`

Modules are file-based and loaded with `import` using a `.ks` path string:

```
import "module::json/myjson.ks"
```

The module's exports are bound to the module namespace specified by its `export` declaration. You can alias the module namespace locally:

```
import "module::json/myjson.ks" as json
value = json.read_json("{\"ok\":true}")
```

Module search paths come from `KANSEI_MODULE_PATH` (colon-separated). If unset, Kansei searches in order:

1) `<main-file-dir>/modules`
2) `/usr/local/lib/kansei/modules`
3) `/usr/lib/kansei/modules`
4) `~/.local/lib/kansei/modules`

Modules are cached and reloaded if the source file changes on disk.

Module exports

Each module must declare a single export header at the top of the file:

```
export demo::mod::sample_mod::[value, add]

value = 3

fn add(a, b)
  a + b
end
```

Exports must refer to local bindings. To re-export, bind locally first:

```
import "demo/mod/sample_mod.ks"
alias_add = demo::mod::sample_mod.add

export demo::mod::reexport::[alias_add]
```

Missing export names are compile errors.

Variables

Variables are dynamically typed and defined on assignment.

```
x = 10
x = "Now a string"
```

Operators

- Arithmetic: `+`, `-`, `*`, `/`
- Comparison: `==`, `!=`, `<`, `>`
- Boolean: `not`, `and`, `&&`, `or`, `||`
- String Concatenation: `"Hello " + "World"`

`and`/`or` are short-circuiting and treat `false`/`nil` as falsey. `&&`/`||` are short-circuiting but require boolean operands.

Control Flow

If / Else

```
if x < 10
  puts "Less than 10"
elif x == 10
  puts "Equal to 10"
else
  puts "Greater than 10"
end
```

While Loop

```
while x > 0
  x = x - 1
end
```

Loop

`loop` repeats a fixed number of times for side effects. You can optionally name the index variable.

```
loop 3
  puts "hi"
end

loop 10 |i|
  puts i
end
```

Collect

`collect` repeats a fixed number of times and returns an array of the block results. Use `into` to fill a pre-allocated array.

```
vals = collect 4 |i|
  i * 2
end
# -> [0, 2, 4, 6]

buf = [0.0; 4]
collect 4 into buf |i|
  i * 2.0
end
```

SIMD note: `simd.*` operations do not require aligned arrays. A `F64Array` is a normal vector of `f64` values with 8-byte alignment. SIMD lanes may be 64-wide (512-byte alignment), so forcing global alignment would increase memory overhead and complicate allocation. The SIMD helpers handle unaligned prefix/suffix elements safely.

For Loop

Iterates over Arrays (values) or Maps (keys).

```
for item in [1, 2, 3]
  puts item
end
```

Functions

Functions are first-class citizens. By default, functions do not implicitly capture outer variables. Explicit reference capture using & is required to access or modify outer bindings.

Definition

```
fn add(a, b)
  a + b # Implicit return of last expression
end
```

Calling

```
res = add(1, 2)
```

Anonymous Functions

Functions can be defined without a name and passed as values.

```
# Using 'fn' keyword
double = fn(x) x * 2 end

# Using block syntax (Closure literal)
triple = {|x| x * 3}
```

```
res = double(5)
```

Currying

Functions support partial application (currying) by default.

```
add10 = add(10) # Returns a new function that takes 1 argument
res = add10(5)  # Returns 15
```

Blocks and Yield

Functions can accept a block of code using `{ |params| ... }`. The function can execute this block using `yield`.

```
fn repeater(n)
  loop n
    yield(i)
  end
end

repeater(3) { |idx|
  puts "Index: " + idx
}
```

Error Handling

Kansei uses `result` to capture runtime errors and `error` to raise them. `result` is an expression and **always requires** an `else`.

Default Fallback

```
y = result { i32(x) } else 1i32
```

Error Handler

```
data = result { IO.read("missing.txt") } else |e| {
  puts f"{e.line}:{e.column} {e.message}"
  puts e.source
  ""
}
```

Rethrow / Wrap

```
result { risky() } else |e| { error e }
```

Notes:

- `result { ... }` catches **runtime** errors only (not syntax errors).
- `result` always requires an `else` clause.
- `else |e| { ... }` requires braces. `else <expr>` is for simple defaults.
- Errors expose `message`, `line`, `column`, `source`, and `trace`.
- `error e` wraps an error, preserving the original trace and adding a new frame.

Variable Scope and References

Kansei enforces strict variable scoping to prevent accidental modification of outer variables.

Implicit Shadowing

Assigning to a variable inside a function or block creates a new local variable, shadowing any outer variable of the same name. Implicit access to outer variables for reading is also restricted in many contexts to ensure isolation.

```
x = 10
fn f()
  x = 20 # Defines a local 'x', does not modify outer 'x'
end
f()
# x is still 10
```

Visibility Keywords (`@file`, `@function`)

By default, functions and variables are only visible in the environment they are defined in. Nested functions do not see outer locals unless explicitly allowed.

- `@file` marks a binding as visible to all functions in the file.
- `@function` marks a binding as visible to all nested functions within the enclosing function scope.
- At top level, `@function` behaves like `@file`.
- `use`, `import`, and `load` can be annotated with `@file` or `@function` to expose their bindings.

```
@file use std::simd # can be on the same line
@file               # but does not have to be
fn helper(x)
  puts std::simd.sum(x)
end

fn outer()
  @function
  x = 3
  @function
  fn inner()
    puts x
  end
  inner()
end
```

Reference Capture (&)

To modify an outer variable or pass a variable by reference, you must use the `&` operator in both the parameter definition and the call site (for functions) or capture list (for blocks).

Functions

```
fn increment(&val)
  val = val + 1
end

count = 0
increment(&count) # Explicit reference passing
# count is now 1
```

Blocks

Blocks passed to functions can also capture outer variables by reference.

```
sum = 0
[1, 2, 3] each { |val, &sum|
  sum = sum + val
}
```

Env Snapshots (%)

`%expr` freezes a Map or Struct into an immutable Env snapshot (deep copy). Use `%{...}` for an Env literal.

```
env = %{ "x": 3 }
env2 = %some_struct
```

Env is read-only; attempts to assign into it will error. To use an Env as an environment, pass it to `std::parallel` or `std::kansei::ast::eval_in`.

Built-in Functions

- `puts(val)`: Print value with newline.
- `print(val)`: Print value without newline.
- `len(obj)`: Return length of String, Array, Map, or Env.
- `read_file(path)`: Read file content as string.
- `write_file(path, content)`: Write string to file.

Format Strings

Prefix a string with `f` to interpolate expressions, similar to Rust formatting.

```
name = "Ada"
count = 3
pi = 3.14159
msg = f"{name} has {count} items"
short_pi = f"{pi:.2}"
```

Use `{{` and `}}` to include literal braces. Precision formatting uses `{expr:.N}`.

Shell Commands

Backticks execute shell commands and capture stdout (trimmed). They also support `{expr}` interpolation (use `{{` and `}}` for literal braces), same as format strings.

```
files = `ls -la`
puts files
name = "Ada"
puts `echo {name}`

## WASM Modules
Use `load wasm::name` to load a WebAssembly module. Kansei searches
`KANSEI_WASM_PATH` (colon-separated). If unset, it looks in:
1) `<main-file-dir>/wasm`
2) `/usr/local/lib/kansei/wasm`
3) `/usr/lib/kansei/wasm`
4) `~/.local/lib/kansei/wasm`

The module is exposed under the `wasm` namespace.

```ruby
load wasm::Json
Json = wasm.json
result = Json.parse(f"{1 + 2}")
```

Select the runtime backend by setting `program.wasm_backend` before loading modules. The default is `"wasmi"`. The available backends are listed in `program.wasm_backends` (for example, `"wasmtime"` only exists when compiled with `--features wasmtime`).

```
program.wasm_backend = "wasmtime"
load wasm::Json
```

WASM ABI

- Export a `memory` and an `alloc(size: i32) -> i32`. `dealloc(ptr: i32, len: i32)` is optional.
```

- For wasm-bindgen modules, `__wbindgen_malloc(size: i32, align: i32) -> i32`, `__wbindgen_free(ptr: i32, len: i32, align: i32)`, and `__wbindgen_add_to_stack_pointer(delta: i32) -> i32` are used when present.
- String arguments are passed as `(i32 ptr, i32 len)` in UTF-8.
- Numeric arguments map to `i32/i64/f32/f64`.
- For string returns, export functions ending with `_str` and return an `i64` where the low 32 bits are `ptr` and high 32 bits are `len` (both `u32`). The host reads from `memory`.
- For wasm-bindgen-style exports that include an initial `i32` retptr parameter (e.g. params are `retptr, ptr, len` and results are `[]` or `[i32]`), the host allocates 8 bytes for `(ptr, len)`, passes that retptr as the first argument, reads the returned `(ptr, len)` from memory, and frees the returned buffer with `dealloc`/`__wbindgen_free` after copying into a Kansei string.

````
### std::IO
The `std::IO` module provides basic file and path utilities:
```ruby
use std::IO
IO = std::IO

IO.write("out.txt", "hello")
IO.append("out.txt", "\nworld")
puts IO.read("out.txt")
bytes = IO.read_bytes("out.txt")
IO.write_bytes("out.bin", bytes)
IO.append_bytes("out.bin", bytes)

puts IO.exists("out.txt")
puts IO.cwd()
IO.mkdirs("tmp/nested")
IO.remove("out.txt")
````

std::lib::clap

`std::lib::clap` provides a small CLI parsing helper:

```
use std::lib::clap
clap = std::lib::clap

args = ["--verbose", "--count", "3", "file.txt"]
flags = ["--verbose"]
options = ["--count"]

parsed = clap.parse(args, flags, options)
puts parsed.verbose
puts parsed.count
puts parsed.args # remaining positional args
```

std::lib::Regex

```
use std::lib::Regex
Regex = std::lib::Regex

puts Regex.is_match("[a-z]+", "abc123")
match = Regex.find("[0-9]+", "abc123")
puts match.match
puts Regex.replace("[0-9]+", "abc123", "###")
parts = Regex.split("\\s+", "a  b   c")
```
```

std::lib::DateTime

```
use std::lib::DateTime
DateTime = std::lib::DateTime

now = DateTime.now_ms
puts DateTime.format(now, "%Y-%m-%d %H:%M:%S")
```

std::lib::Crypto

```
use std::lib::Crypto
Crypto = std::lib::Crypto

puts Crypto.sha256("hello")
puts Crypto.blake3("hello")
puts Crypto.hmac_sha256("key", "msg")
puts Crypto.random_bytes(16)
rand_bytes = Crypto.random_bytes_buf(16)
```

std::lib::Http

```
use std::lib::Http
Http = std::lib::Http

resp = Http.get("https://example.com")
puts resp.status
puts resp.body
```

std::lib::Csv

```
use std::lib::Csv
Csv = std::lib::Csv

rows = Csv.parse("a,b\n1,2\n")
puts rows
text = Csv.stringify(rows)
```

std::lib::Path

```
use std::lib::Path
Path = std::lib::Path

puts Path.join("/tmp", "file.txt")
puts Path.basename("/tmp/file.txt")
puts Path.dirname("/tmp/file.txt")
puts Path.ext("/tmp/file.txt")
```

std::lib::Math

```
use std::lib::Math
Math = std::lib::Math

puts Math.sin(1.0)
puts Math.pow(2, 8)
```

std::lib::Bytes

```
use std::lib::Bytes
Bytes = std::lib::Bytes

buf = Bytes.buf(4, 0)
Bytes.set(buf, 0, 255)
Bytes.push(buf, 1)
```

```
Bytes.fill(buf, 3)
Bytes.copy(buf, 1, Bytes.from_string("hi"), 0, 2)
bytes = Bytes.freeze(buf)
puts Bytes.len(bytes)
puts Bytes.to_string(Bytes.from_string("hello"))
puts Bytes.find(Bytes.from_string("hello"), Bytes.from_string("ell"))
view = Bytes.slice_view(bytes, 0, 2)
```

std::lib::Net

```
use std::lib::Net
Net = std::lib::Net

conn = Net.connect("imap.example.com", 993, true)
conn.write("NOOP\r\n")
line = conn.read_line(4096)
conn.close()
```

std::lib::Tui

```
use std::lib::Tui
Tui = std::lib::Tui

Tui.run(16, |ui, event| {
  size = ui.size()
  ui.paragraph(0, 0, size.width, 3, "Hello from Kansei", "Header")
  if event.type == "key" && event.key.code == "Esc" { false } else { true }
})
```

std::lib::Mmap

```
use std::lib::Mmap
use std::lib::Bytes
Mmap = std::lib::Mmap
Bytes = std::lib::Bytes

map = Mmap.open("data.bin", "r")
chunk = Mmap.read(map, 0, 16)
puts Bytes.len(chunk)
view = Bytes.slice_view(map, 0, 16)
```

std::lib::Polars

```
use std::lib::Polars
Polars = std::lib::Polars

df = Polars.read_csv("data.csv")
puts Polars.shape(df)
puts Polars.columns(df)
```

std::lib::Serde

```
use std::lib::Serde
Serde = std::lib::Serde

data = Serde.parse("{\"ok\":true}")
puts Serde.stringify(data)
```

std::lib::Base64

```
use std::lib::Base64
Base64 = std::lib::Base64
```

```
encoded = Base64.encode("hello")
puts Base64.decode(encoded)
bytes = Base64.decode_bytes(encoded)
array = Base64.decode_array(encoded)
```

std::lib::Uuid

```
use std::lib::Uuid
Uuid = std::lib::Uuid

id = Uuid.v4()
puts Uuid.is_valid(id)
```

std::lib::Toml

```
use std::lib::Toml
Toml = std::lib::Toml

val = Toml.parse("a = 1")
puts Toml.stringify(val)
```

std::lib::Yaml

```
use std::lib::Yaml
Yaml = std::lib::Yaml

val = Yaml.parse("a: 1")
puts Yaml.stringify(val)
```

std::lib::Flate2

```
use std::lib::Flate2
Flate2 = std::lib::Flate2

compressed = Flate2.compress("hello")
puts Flate2.decompress(compressed)
```

std::lib::Image

```
use std::lib::Image
Image = std::lib::Image

img = Image.load_png("in.png")
puts img.width
Image.save_png("out.png", img.width, img.height, img.rgba)

img_bytes = Image.load_png_bytes("in.png")
Image.save_png_bytes("out.png", img_bytes.width, img_bytes.height, img_bytes.rgba)
```

std::lib::Sqlite

```
use std::lib::Sqlite
Sqlite = std::lib::Sqlite

db = Sqlite.open("example.db")
Sqlite.exec(db, "create table if not exists items (id integer, name text)")
Sqlite.exec(db, "insert into items values (1, 'a')")
rows = Sqlite.query(db, "select id, name from items")
rows_bytes = Sqlite.query_bytes(db, "select id, name from items")
puts rows
```